

FocusAI Retrieval Map

Everything the agent can reach across Canvas and your notes, the order it has to reach it in, and the exact fields that come back.

11 MCP tools 2 data worlds Claude Haiku 4.5 · LangGraph ReAct
read-only, except `download_files`

OVERVIEW

Two worlds, reached through two different doors

The agent serves one chat panel but pulls from two unrelated stores. Canvas is somebody else's system, deep and hierarchical, reached with a token. Notes are the user's own writing, flat and shallow, reached over the same REST API the browser uses. Every workflow in this document is one or the other; nothing joins them except the answer the agent writes at the end.

WORLD A · CANVAS LMS

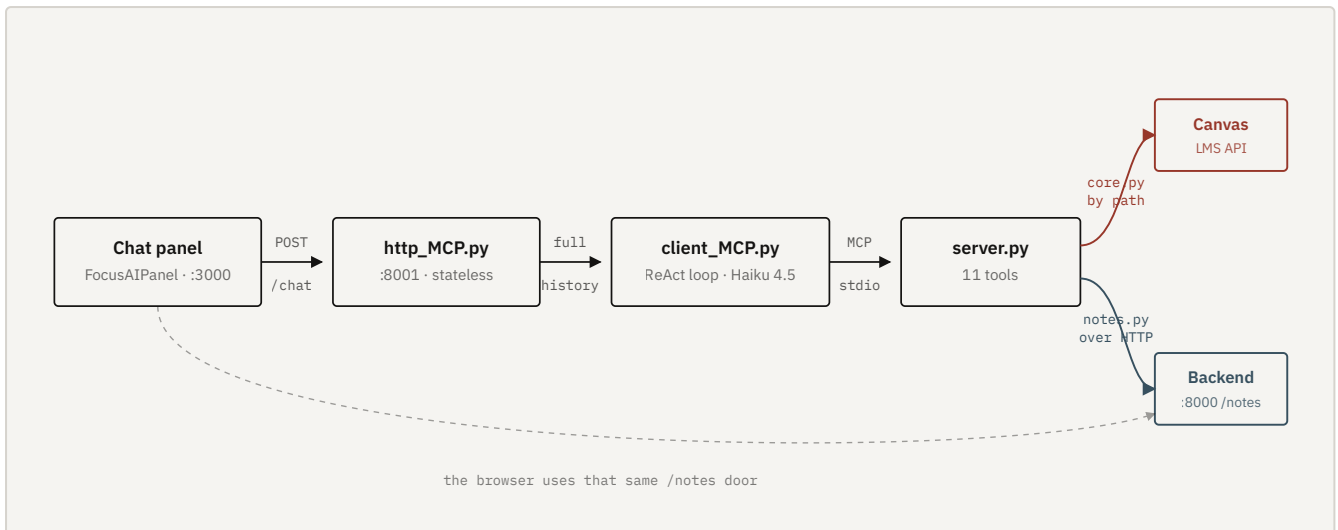
9 tools, 4 levels deep

A tree. Nothing below the top level is reachable without first resolving a `course_id`, and file bytes are three hops past that.

WORLD B · YOUR NOTES

2 tools, 1 step apart

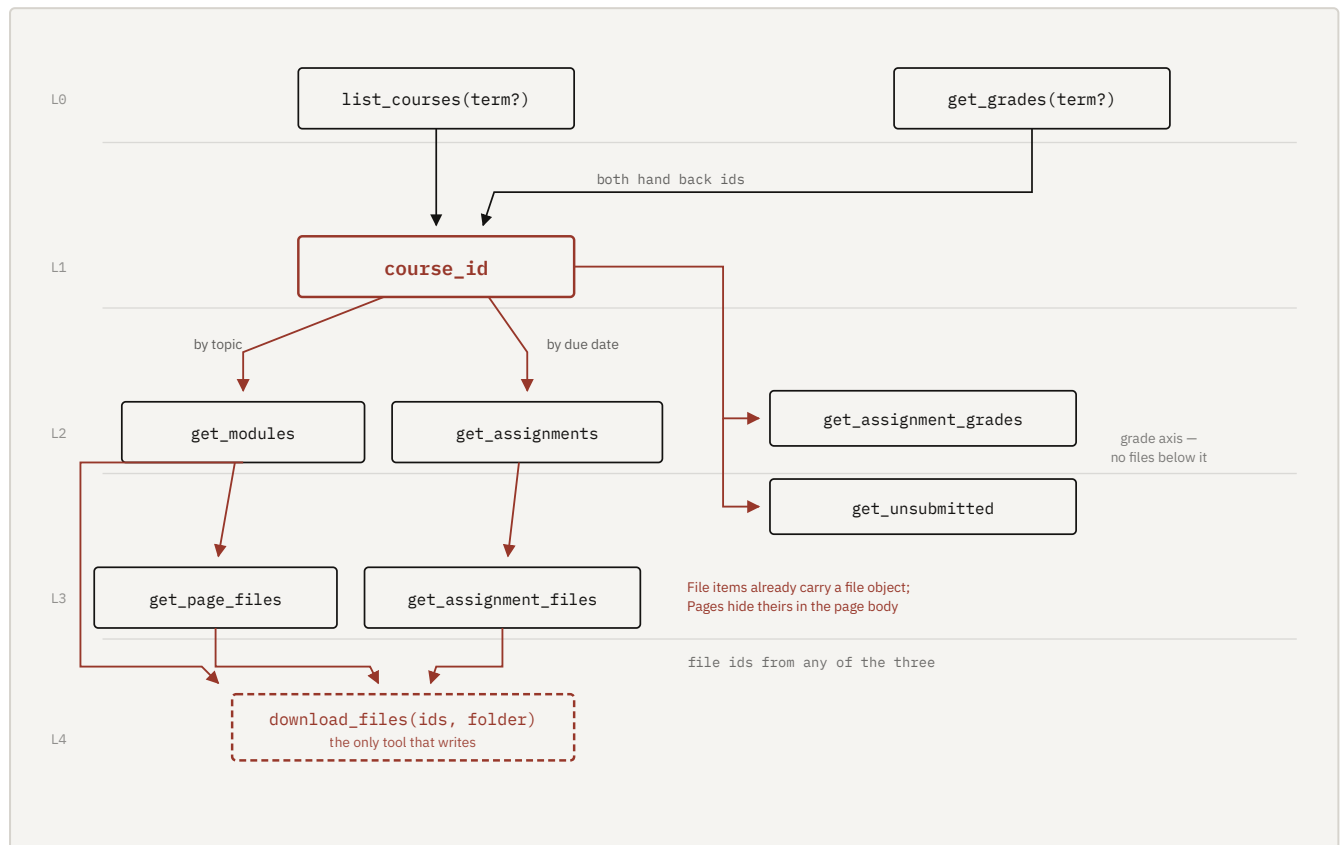
A list and a reader. No tree, no search — the ids exist in exactly one place, and that place has to be visited first.



The agent is a peer of the browser, not a privileged insider. Canvas is reached by loading `backend/apis/canvas/core.py` off disk by file path — one implementation, no web framework on the agent side. Notes are reached over HTTP through the identical `/notes` endpoints the To-Do page calls. The chat service keeps no session: the panel owns the transcript and resends all of it with every message.

The Canvas hierarchy

Canvas has one key that opens everything: the **course id**. It is not guessable, it is never in the user's question, and no tool below the top level works without it. That single fact sets the shape of every Canvas workflow — the first call is always a lookup, and the real work starts on the second.



Two axes hang off one id, and only one of them reaches files. `get_grades` is a second top-level entry — it takes a term, not a course id, and returns ids too, so a grades question can skip `list_courses` entirely. The dashed leaf is the only tool in the whole server that changes anything on disk.

L0 Term — the only thing filterable without an id

`list_courses(term)` and `get_grades(term)` take a substring like "2026 Spring" and are the two doors into everything else. Both include `unpublished`, `available` and `completed` states, so next semester's courses appear here before an instructor publishes them, and a course from years ago is still reachable.

L1

Course — the key, and the naming trap

Every course carries two names, and Canvas stores them the opposite way round to what the field names imply: `name` is the section code (2026S CS 334-A) and `title` is the readable one (Theory of Computation). The agent matches user wording against *both*, case-insensitively, on substrings — people say the title or a fragment, almost never the code.

L2

Structure — the same course, cut two ways

`get_modules` groups by topic and holds the material. `get_assignments` groups by time into Upcoming / Undated / Past and describes the workload. "Where is X" goes left; "what is due" goes right. Undated is not filler — rolled-up marks like *Homework average* live there with no due date.

L3

Content — where the file ids actually appear

Module items are a **flat ordered list**, not a tree: nesting lives in `indent`, which is what the Canvas web page renders as indentation. An `indent=1` File under an `indent=0` Assignment is that assignment's attachment. Only `File` items carry a `file` object; `Page`, `Assignment` and `ExternalUrl` items have `file: null` and are reached through `link`.

L4

Bytes — and why the agent usually skips this level

Every file object already carries a **pre-signed url**, so the normal answer is a Markdown link the panel renders as a clickable chip. `download_files` writes to `~/Downloads/<folder>` on the machine running the agent — under Docker that is inside a container the user cannot open. It is called only when someone explicitly asks for files saved to disk.

What each Canvas tool returns

Nine tools, and the exact field names that come back. Anything not in these tables, the agent cannot state — its prompt forbids it from inventing a number, filename, date or grade a tool did not return.

Finding and structuring

TOOL	TAKES	GIVES BACK
list_courses	term <i>optional</i>	id · name · title · term <i>name is the section code, title is the readable one</i>
get_modules	course_id	module · items[type · title · indent · link · file] <i>file, when present: id · name · type · kb · url</i>
get_assignments	course_id	bucket · assignments[name · group · due · score · possible · state] <i>group is the weighted category: Homework, Exams, Attendance</i>

Reaching files

TOOL	TAKES	GIVES BACK
get_page_files	course_id page <i>optional</i>	module · page · files[id · name · type · kb · url] <i>where lecture slides live: inside Page bodies, not module items</i>
get_assignment_files	course_id	assignment · files[id · name · type · kb · url] · links[] <i>scraped out of assignment descriptions</i>
download_files	file_ids folder	name · kb · path or id · error <i>writes to disk: the only non-read tool in the server</i>

Grades

TOOL	TAKES	GIVES BACK
<code>get_grades</code>	<code>term</code> <i>optional</i>	<code>id · name · title · term · score · letter</code>
<code>get_assignment_grades</code>	<code>course_id</code> <code>graded_only</code>	<code>name · score · possible · state · due</code>
<code>get_unsubmitted</code>	<code>course_id</code>	<code>name · possible · due</code> <i>filtered by Canvas server-side</i>

Five things the field names do not tell you

Lecture slides are invisible to `get_modules`

A lecture shows up as a Page item with `file: null`, because its PDFs are links written into the page body rather than module items. Asked for slides, `get_modules` alone makes the whole course look like it has no files — `get_page_files` fetches each page and scrapes it.

Solution keys go the other way

They are published almost only as File items inside modules, not attached to the assignment. A solutions question needs `get_modules`, never `get_assignment_files`.

One lecture usually publishes three PDFs

`*.short.pdf` handout without worked solutions · `*.pdf` full slides with sample solutions · `*.marked.pdf` full slides plus notes written in class. If the user did not say which, the agent asks rather than silently returning all three.

A score of `null` is not a zero

Canvas scores are grade-to-date: ungraded work is not counted against you, so an early-semester percentage is not predictive. Likewise *unsubmitted* includes work that is not due yet — it is not a list of what you are late on.

Narrow the page filter, or pay for it

`page` is a plain substring, so `"Lecture 1"` also matches Lectures 10 through 19 — `"Lecture 1:"` pins one. Omitting it sweeps every page in the course, roughly 30 seconds of Canvas calls.

Canvas workflows, end to end

Six questions the panel actually gets, and the call chain each one resolves into. Every chain starts with a name-to-id lookup, because there is no other way to get an id.

"Give me the Lecture 3 slides for theory of comp"

1. `list_courses()` — match "theory of comp" against name and title → `course_id`
2. `get_page_files(course_id, page="Lecture 3")` → module, page title, and the file objects with ids and pre-signed urls
3. If all three variants came back, ask which — handout, full, or marked

Answer: one Markdown link per file, rendered in the panel as a download chip. No download call.

"What's due this week?"

1. `list_courses(term)` → every course id for the term
2. `get_assignments(course_id)` **once per course** — there is no cross-course due-date tool on the agent side
3. Read the `Upcoming` bucket from each and merge by `due`

Cost scales with course count. The Home page's task ring gets the same answer in one shot from `GET /canvas/tasks`, a REST route the agent does not have.

"How am I doing this semester?"

1. `get_grades(term)` — one call, no `list_courses` needed; it returns ids as well as scores
2. Drill into any course with `get_assignment_grades(course_id)`

"This semester" resolves to the most recent term that actually has grades — a term that just started has none — and the agent says which term it used.

"What haven't I turned in for 334?"

1. `list_courses()` → match on the "334" fragment
2. `get_unsubmitted(course_id)` → name, points possible, due date

An empty list means nothing is outstanding, not that the lookup failed — and the agent reports it that way.

"Where's the homework 2 handout?"

1. `list_courses()` → `course_id`
2. `get_assignment_files(course_id)` → the PDF with the actual questions, plus starter files and outside links

These come out of assignment descriptions, because `/courses/{id}/files` returns 403 whenever an instructor hides the Files tab from students.

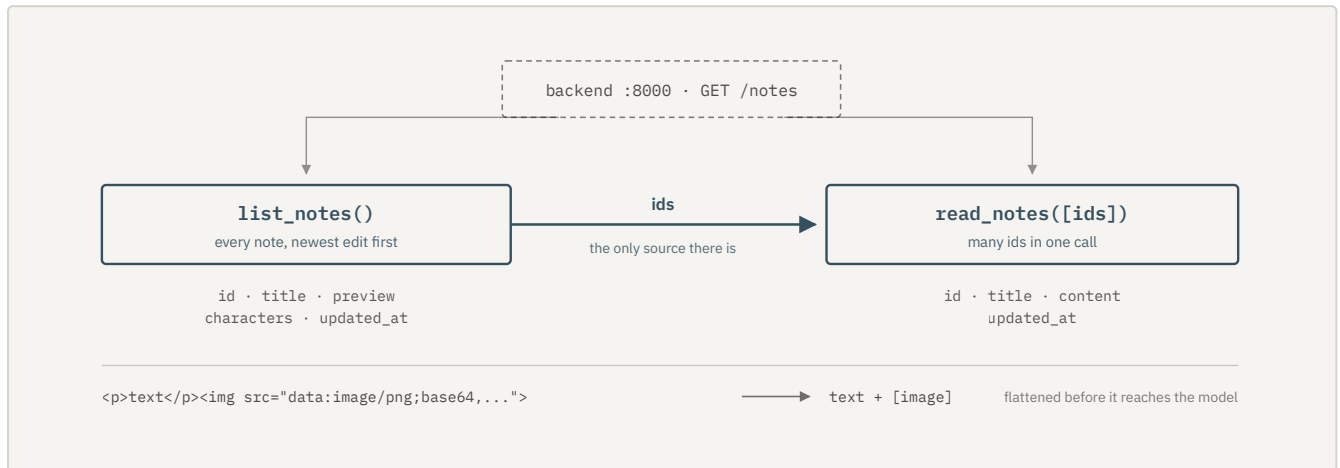
"Save lectures 1 to 3 into a folder called 334"

1. `list_courses()` → `course_id`
2. `get_page_files` three times, or once unfiltered → collect the `file.id` values
3. `download_files(file_ids, folder="334")` → names, sizes and full paths written

Only on an explicit save-to-disk request. `folder` is one bare name, sanitised against traversal; nothing can be written outside `~/Downloads` .

Notes: a two-step chain, and no third step

Notes have no tree, no folders, no search endpoint and no link to a course. There are two tools, and the second is unusable until the first has run — note ids are not in the title, the user does not know them, and one the model invents either fails or quietly reads back a different note.



Both tools read the same REST endpoints, but only the first one produces ids. Note bodies are stored as HTML by the contentEditable editor, and a pasted screenshot lands in there as a base64 data URI — so the server flattens to plain text and throws the bytes away, which the model could not read in any case.

s1 **list_notes()** — no arguments, always first

Returns every note, most recently edited first: id · title · preview · characters · updated_at. Titles are optional in the editor, so an untitled note comes back as (untitled) and the 160-character preview is the only thing identifying it. Matching runs on substrings, case-insensitively, over the preview as well as the title.

s2 **read_notes([ids])** — batched, and forgiving of one bad id

Takes several ids at once, so a question spanning three notes is one call rather than three. An id with nothing behind it comes back carrying **error** instead of content, and the rest are still returned. When two or three notes could be the one meant, the agent reads them all and answers from whichever fits — asking the user to disambiguate is the last resort, not the first move.

"What did I write about the automata proof?"

1. `list_notes()` → scan titles *and* previews for `automata`
2. `read_notes([3, 7])` — both candidates in one call
3. Answer from the body that actually fits, and flag any `[image]` marker sitting where the proof was

The agent answers only from what a note literally says. A pasted screenshot is unavailable to it, so it reports that a picture is there rather than guessing at what it showed.

The rules the agent runs under

Each of these is enforced twice — once in the system prompt the model sees on every turn, and once in the tool docstrings, which are the only channel it gets while choosing a call.

Every id comes from a tool result

Never from the user, never from a guess, never carried over from an earlier question about something else. This holds for course ids, file ids and note ids alike.

Names are matched on substrings, against every name field

Courses carry two names and the readable one is not where the field name suggests. If several match, the agent asks which, listing title, code and term. If none match, it says so and shows what it did find, rather than concluding the course does not exist.

Files are handed over as links, not fetched

The reply is read in a browser panel that renders Markdown, so a file becomes `[L1.Automata.pdf](url)` (1183 KB) . That url is pre-signed and expires, so the agent gives it out on discovery rather than describing the file and waiting to be asked.

Nothing is stated that a tool did not return

No number, filename, date or grade gets invented. An empty tool result is reported as empty rather than filled in.

The conversation lives in the browser

`POST /chat` keeps no session — the panel owns the transcript and resends the whole history with every message, so two tabs asking at once never see each other's conversation. The LangGraph agent itself is built once and reused behind a lock, so no message pays a fresh MCP server start-up.

What it cannot retrieve

Worth knowing before asking. None of these are bugs; they are the edges of the eleven tools.

-
- **No cross-course view in one call.** Every structure and grade tool below the top level takes a single `course_id`. "What's due everywhere this week" is a fan-out, one call per course. The REST route `GET /canvas/tasks` does it in one pass for the Home page, but it is not exposed as an MCP tool.

 - **No writing to Canvas.** No submitting, no commenting, no uploading, no calendar edits. Every Canvas tool is a GET; the one write in the whole server puts files in `~/Downloads`.

 - **No creating or editing notes.** The backend has `POST`, `PATCH` and `DELETE` on `/notes`, and the To-Do page uses all three — but the agent is wired only to the two read paths.

 - **No reading a file's contents.** The tools return a PDF's name, type, size and url. Nothing opens it, so the agent cannot answer a question about what is *inside* a lecture deck — only point at it.

 - **No images.** A screenshot pasted into a note arrives as the marker `[image]`, with the bytes discarded before the model ever sees them.

 - **No note search.** There is no query endpoint; finding a note means listing all of them and matching on title and preview in the model's head.

 - **No files a hidden Files tab would have shown.** Attachments are recovered by scraping `/files/{id}` links out of page and assignment HTML. A file linked no other way, or one this Canvas token cannot read, is silently absent rather than reported as an error.

 - **One page of results per call.** Canvas requests ask for 100 items and do not follow pagination links, so a collection larger than 100 — courses on the account, assignments in one course, items in one module — is cut off at that ceiling.

 - **Nothing from the rest of FocusLab.** Spotify, queues, the Pomodoro timer and the calendar have no tools. The agent sees Canvas and notes, and nothing else in the app.
